Assignment 1 - Design Decisions Report

Anoop Singh (2023CS10459)

January 27, 2026

1 Introduction

This report documents the design decisions and performance considerations for the parallel implementation of three core functions in the order book processing system: `updateDisplay`, `totalAmountTraded`, and `printOrderStats`. Each function has been optimized using OpenMP parallel programming techniques to improve performance over their sequential counterparts.

2 Function Analysis

2.1 updateDisplay Function

Design Decisions: The parallel implementation uses a multi-phase approach: (1) parallel decode and count, (2) prefix-sum for grouping, (3) parallel processing, and (4) parallel file writing.

A compact `SnapshotStock` struct (3 bytes: 2 for stockID, 1 for spread) minimizes memory footprint. Thread-local counting arrays avoid contention during the counting phase. Stock entries are grouped using parallel prefix sums, and each snapshot's data is processed independently. Output uses buffered `FILE*` with `fwrite` for fast I/O, with parallel snapshot file generation.

2.2 totalAmountTraded Function

Design Decisions: The `totalAmountTraded` function implements a straightforward parallel reduction pattern using OpenMP's built-in reduction clause. The design prioritizes simplicity and efficiency by leveraging the `#pragma omp parallel for reduction(+:total)` directive.

2.3 printOrderStats Function

Design Decisions: Each thread maintains its own section of a flat array (`allThreadStats[thread_id * MAX_STOCKS]`) using a compact `StockStatsCompact` struct. Since stock IDs are bounded ($\leq$100,000), fixed arrays provide O(1) access without hash overhead. After parallel processing, the merge phase is also parallelized across stock IDs using `#pragma omp parallel for`, with chunking (`schedule(static, 2048)`) for cache efficiency. Output uses buffered `FILE*` with `fwrite`.

3 Scalibilty

3.1 speedup vs thread graph

The performance of the parallel implementations was evaluated by measuring the speedup achieved with varying numbers of threads. The program was run on an orderbook having 1 million entries

and snapshots taken every 10000 entries, and the execution time was recorded for different thread counts. The results are summarized in Figure 1.

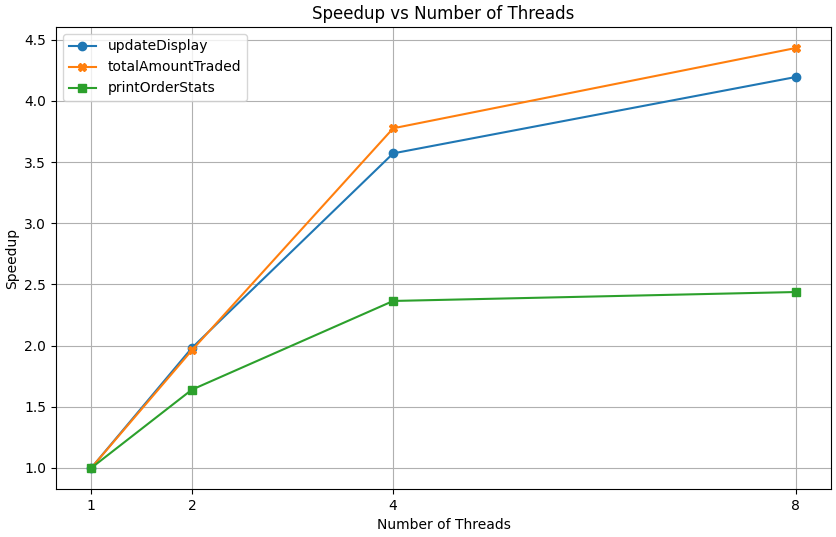


Figure 1: Speedup vs Number of Threads

4 Conclusion

The parallel implementations demonstrate effective use of OpenMP constructs to achieve significant performance improvements over sequential versions. The design decisions prioritize correctness, minimize synchronization overhead, and maximize parallel efficiency while maintaining the functional requirements of the order book processing system.